Course Title: Data Base Management System

Semester: V

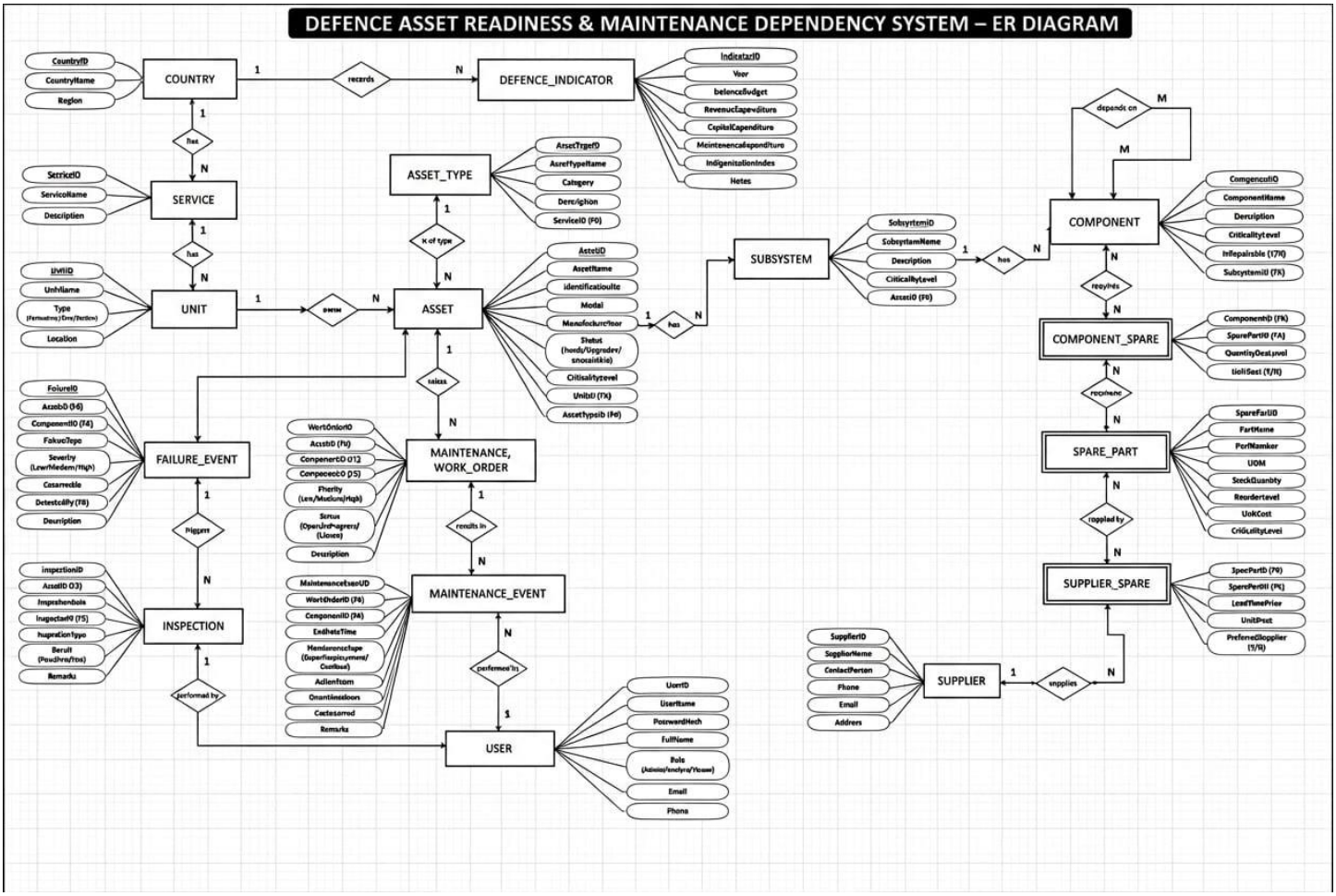
Academic Session: Aug-Dec 2026

DBMS Experiential Learning – Level 3 Project

Name:	SRN:	Section:
Raksha	PES2UG24CS652	K
Praneeth P Shetty	PES2UG24CS902	K

Title : Defence Asset Readiness and Maintanance Dependency System

ER diagram:



Relational Schema:



DDL Commands:

```sql
CREATE DATABASE IF NOT EXISTS defence_asset_system;
USE defence_asset_system;


-- 1. COUNTRY

CREATE TABLE COUNTRY (
    CountryID INT PRIMARY KEY,
    CountryName VARCHAR(100) NOT NULL,
    Region VARCHAR(100)
);


-- 2. USER

CREATE TABLE `USER` (
    UserID INT PRIMARY KEY,
    UserName VARCHAR(50) NOT NULL,
    PasswordHash VARCHAR(255) NOT NULL,
    FullName VARCHAR(100),
    Role VARCHAR(30),
    Email VARCHAR(100),
    Phone VARCHAR(20)
);


-- 3. SERVICE

CREATE TABLE SERVICE (
    ServiceID INT PRIMARY KEY,
    ServiceName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CountryID INT NOT NULL,

    FOREIGN KEY (CountryID)
        REFERENCES COUNTRY(CountryID)
);


-- 4. DEFENCE_INDICATOR

CREATE TABLE DEFENCE_INDICATOR (
    IndicatorID INT PRIMARY KEY,
    CountryID INT NOT NULL,
    Year INT NOT NULL,
    DefenceBudget DECIMAL(15,2),
    RevenueExpenditure DECIMAL(15,2),
    CapitalExp DECIMAL(15,2),
    MaintenanceExp DECIMAL(15,2),
```

```
    IndigenisationIndex DECIMAL(5,2),
    Notes VARCHAR(500),

    FOREIGN KEY (CountryID)
        REFERENCES COUNTRY(CountryID)
);


-- 5. UNIT

CREATE TABLE UNIT (
    UnitID INT PRIMARY KEY,
    UnitName VARCHAR(100) NOT NULL,
    Type VARCHAR(50),
    Location VARCHAR(100),
    ServiceID INT NOT NULL,

    FOREIGN KEY (ServiceID)
        REFERENCES SERVICE(ServiceID)
);


-- 6. ASSET_TYPE

CREATE TABLE ASSET_TYPE (
    AssetTypeID INT PRIMARY KEY,
    AssetTypeName VARCHAR(100) NOT NULL,
    Category VARCHAR(100),
    Description VARCHAR(255)
);


-- 7. ASSET

CREATE TABLE ASSET (
    AssetID INT PRIMARY KEY,
    AssetName VARCHAR(100) NOT NULL,
    IdentificationNo VARCHAR(100),
    Model VARCHAR(100),
    MnfYear INT,
    Status VARCHAR(30),
    CriticalityLevel VARCHAR(20),
    UnitID INT NOT NULL,
    AssetTypeID INT NOT NULL,

    FOREIGN KEY (UnitID)
        REFERENCES UNIT(UnitID),

    FOREIGN KEY (AssetTypeID)
        REFERENCES ASSET_TYPE(AssetTypeID)
);
```

```sql
-- 8. FAILURE_EVENT

CREATE TABLE FAILURE_EVENT (
    FailureID INT PRIMARY KEY,
    FailureType VARCHAR(100),
    Severity VARCHAR(20),
    OccurredAt DATETIME,
    Description VARCHAR(500),
    AssetID INT NOT NULL,

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);


-- 9. INSPECTION

CREATE TABLE INSPECTION (
    InspectionID INT PRIMARY KEY,
    FailureID INT NOT NULL,
    InspectorID INT NOT NULL,
    InspectionType VARCHAR(100),
    InspectionDate DATE,
    Result VARCHAR(50),
    Remarks VARCHAR(500),

    FOREIGN KEY (FailureID)
        REFERENCES FAILURE_EVENT(FailureID),

    FOREIGN KEY (InspectorID)
        REFERENCES `USER`(UserID)
);


-- 10. MAINTENANCE_WORK_ORDER

CREATE TABLE MAINTENANCE_WORK_ORDER (
    WorkOrderID INT PRIMARY KEY,
    AssetID INT NOT NULL,
    Description VARCHAR(500),
    OrderDate DATE,
    Priority VARCHAR(20),
    Status VARCHAR(30),

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);
```

```sql
-- 11. MAINTENANCE_EVENT

CREATE TABLE MAINTENANCE_EVENT (
    MaintenanceID INT PRIMARY KEY,
    WorkOrderID INT NOT NULL,
    Remarks VARCHAR(500),
    PerformedBy INT NOT NULL,
    StartDateTime DATETIME,
    EndDateTime DATETIME,
    MaintenanceType VARCHAR(50),
    ActionTaken VARCHAR(500),
    DowntimeHours DECIMAL(10,2),
    CostIncurred DECIMAL(15,2),

    FOREIGN KEY (WorkOrderID)
        REFERENCES MAINTENANCE_WORK_ORDER(WorkOrderID),

    FOREIGN KEY (PerformedBy)
        REFERENCES `USER`(UserID)
);


-- 12. SUBSYSTEM

CREATE TABLE SUBSYSTEM (
    SubsystemID INT PRIMARY KEY,
    SubsystemName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CriticalityLevel VARCHAR(20),
    AssetID INT NOT NULL,

    FOREIGN KEY (AssetID)
        REFERENCES ASSET(AssetID)
);


-- 13. COMPONENT

CREATE TABLE COMPONENT (
    ComponentID INT PRIMARY KEY,
    ComponentName VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    CriticalityLevel VARCHAR(20),
    IsRepairable VARCHAR(5),
    SubsystemID INT NOT NULL,

    FOREIGN KEY (SubsystemID)
        REFERENCES SUBSYSTEM(SubsystemID)
);
```

```sql
-- 14. COMPONENT_DEPENDENCY
-- Recursive relationship
-- Both FKs reference COMPONENT

CREATE TABLE COMPONENT_DEPENDENCY (
    ComponentID INT NOT NULL,
    DependsOnComponentID INT NOT NULL,

    PRIMARY KEY (ComponentID, DependsOnComponentID),

    FOREIGN KEY (ComponentID)
        REFERENCES COMPONENT(ComponentID),

    FOREIGN KEY (DependsOnComponentID)
        REFERENCES COMPONENT(ComponentID),

    CHECK (ComponentID <> DependsOnComponentID)
);


-- 15. SPARE_PART

CREATE TABLE SPARE_PART (
    SparePartID INT PRIMARY KEY,
    PartName VARCHAR(100) NOT NULL,
    PartNumber VARCHAR(100),
    UOM VARCHAR(30),
    StockQuantity INT DEFAULT 0,
    ReorderLevel INT DEFAULT 0,
    UnitCost DECIMAL(12,2),
    CriticalityLevel VARCHAR(20)
);


-- 16. COMPONENT_SPARE
-- M:N relationship between COMPONENT and SPARE_PART

CREATE TABLE COMPONENT_SPARE (
    ComponentID INT NOT NULL,
    SparePartID INT NOT NULL,
    QuantityRequired INT,
    IsCritical VARCHAR(5),

    PRIMARY KEY (ComponentID, SparePartID),

    FOREIGN KEY (ComponentID)
        REFERENCES COMPONENT(ComponentID),

    FOREIGN KEY (SparePartID)
        REFERENCES SPARE_PART(SparePartID)
```

```sql
);


-- 17. SUPPLIER

CREATE TABLE SUPPLIER (
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(100) NOT NULL,
    ContactPerson VARCHAR(100),
    Phone VARCHAR(20),
    Email VARCHAR(100),
    Address VARCHAR(255)
);


-- 18. SUPPLIER_SPARE
-- M:N relationship between SUPPLIER and SPARE_PART

CREATE TABLE SUPPLIER_SPARE (
    SupplierID INT NOT NULL,
    SparePartID INT NOT NULL,
    LeadTimeDay INT,
    UnitPrice DECIMAL(12,2),
    PreferredSupplier VARCHAR(5),

    PRIMARY KEY (SupplierID, SparePartID),

    FOREIGN KEY (SupplierID)
        REFERENCES SUPPLIER(SupplierID),

    FOREIGN KEY (SparePartID)
        REFERENCES SPARE_PART(SparePartID)
);
```